

Internet scanning and service discovery

Follow-along: find the hosts, identify the services, recover a login

Overview

Definition: Reconnaissance

The stage of an attack that runs before anything is exploited: establishing which addresses in a target prefix carry a host that replies to a given probe, which transport-layer ports on those hosts have a process listening, and which software is behind each listener. A host that replies to nothing the attacker sends has not been shown to be absent, only to be unmeasured by the probes used. Reconnaissance produces a target list, not access.

An organisation's allocated prefix is frequently a matter of public record, published in the address registry's registration data and visible in the global routing table. How that prefix is divided internally is not. A /18 may be subnetted into dozens of smaller networks, and which addresses inside them are assigned to a running host, and which of those hosts have a service listening, is what an attacker has to measure. **Reconnaissance** is that measurement, and it is carried out by sending probes whose replies are diagnostic: an ICMP echo request drawing an echo reply, a TCP SYN drawing a SYN-ACK from a listening port or a reset from a closed one, a UDP datagram drawing either a reply from the service itself or an ICMP port-unreachable message from the host. Each probe tests one protocol, and for TCP and UDP one port, so each result is a lower bound on what is present rather than an inventory of it.

This lab gives you one /24 and nothing else about it, and has you map it in three passes: identify the addresses that reply, identify the software listening on each, then use what that yields to obtain a login on one of them. Each pass narrows the one before it. Only part of the prefix replies, only part of what replies has a listener, and one of those listeners discloses enough about a second host to attack an account on it. The probes used here are ICMP and TCP; UDP services such as SNMP, NTP and DNS over UDP, and anything reached over QUIC, reply to nothing this lab sends, and a survey that had to account for them would need separate probes for each. Detecting a scan like this, and blocking it, is a separate lab; nothing here defends anything.

Running this lab in the TUI

You drive the lab from the MiniLabs launcher. Clone it if the machine does not have it, and pull if it does, because handouts and lab scripts change together:

```
git clone https://github.com/nzRichie/MiniLab.git ~/MiniLab # first time
cd ~/MiniLab && git pull                                     # already cloned
```

Start the launcher from that directory:

```
cd ~/MiniLab && ./minilabs
```

If it exits with a Docker error, follow the setup in the repository's `README.md` and run it again.

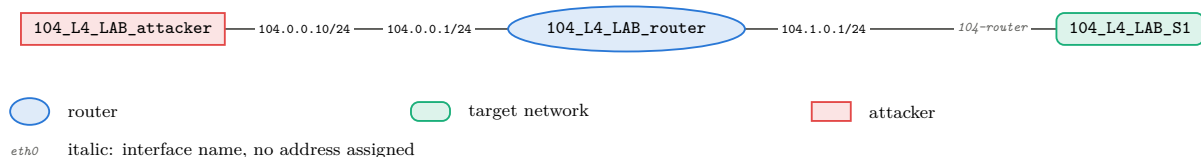
The root menu lists one category per row. Select **Layer 3 and 4 attacks and mitigations**, then **Internet scanning and service discovery**, to reach this lab's actions:

- **Spawn** creates this lab's containers and wiring: the machine you work from, the router, and the network you are asked to survey. Wait for the panel to turn green before you go on. The first spawn of a lab also builds its images, which takes a few minutes; later spawns take seconds.
- **Status** prints the addressing you have been given and confirms each service in the lab is running and reachable. It deliberately does not tell you which addresses those services sit on, because working that out is the exercise. Status takes over the whole screen: the arrow keys scroll its output, and Enter on **Done** closes it.
- **Shell command for a node** lists every node and the `docker exec -it ...` line that opens a shell on it. Copy the line for the node a step below names into your own terminal. It holds the screen until Enter on **Done**, as Status does. Every step below runs on **attacker**; the other nodes are listed so you can look at a host after you have found it, not before.
- **Reset to baseline** deletes the scan output you have accumulated and restarts the services, so you can run a part again from nothing.
- **Teardown** removes the lab.
- **Open handout** opens this document.

Spawn the lab now, then work through Part 1 in your own terminal. **Back** (or Esc) returns to the lab list.

The setup

You work from one machine, on the far side of a router from the network you are surveying. You have no account on it, no interface inside it, and no inventory of it; the single piece of information you have been given is the prefix it occupies.



Nothing is drawn inside the target network because nothing about it has been given to you. What you do have:

What	Value	Container
your machine	104.0.0.10	104_L4_LAB_attacker
your interface	104-ext	
your gateway	104.0.0.1	104_L4_LAB_router
the target prefix	104.1.0.0/24	

The router forwards between your subnet and the target prefix and filters nothing, so every probe you send arrives and every reply comes back. Which addresses inside 104.1.0.0/24 are assigned to a host, and which software each of those hosts is running, are what Parts 1 and 2 have you establish.

Your machine carries stock tooling; there is no bespoke exploit anywhere in this lab. Each tool gets a definition box at the step that first uses it, listing the arguments this lab hands it:

- three scanners: **zmap**, **masscan** and **nmap**;

- **hydra**, for guessing a password against a login service;
- **curl**, **ncat**, **telnet** and **tcpdump**, for connecting by hand and reading what actually crosses the wire;
- the shell's own **grep**, **awk**, **tee**, **cat**, **head** and **wc**, for reducing scanner output to the fields a later step reads.

Definition: **docker exec**

Runs one command inside a container that is already running. **-i** keeps your keyboard attached to that command and **-t** gives it a terminal; written together as **-it**, they hand you an interactive shell rather than running one command and exiting. The name after the flags picks the container; the rest of the line is the command to run inside it, here **bash**.

Open a shell on your machine with **docker exec** and stay there; every command in this handout runs on it:

```
# your own terminal: this is the line that opens the shell
docker exec -it 104_L4_LAB_attacker bash
```

Assessment. This lab is graded on eighteen questions, marked inline below as **Question *N***, each placed at the step whose output it asks about. Answer all eighteen in a separate document as you work through the steps. Every question is answered from what you see on screen at that step.

Part 1: which addresses hold a host

Definition: **Host discovery (ping sweep)**

Sending one probe to every address in a prefix and recording which addresses reply. The probe the name refers to is an ICMP echo request, the same one **ping** sends, which is why the technique is called a ping sweep; the same sweep can be run with a TCP SYN or any other probe type, and each choice tests a different property. **104.1.0.0/24** contains 256 addresses, of which 254 can be assigned to an interface: the first is the network address and the last is the subnet broadcast address. A scanner handed the block in CIDR form probes all 256, because it treats the argument as a range of addresses rather than as a subnet with reserved ends, so a sweep of this /24 sends 256 probes and prints one line per address that replies.

Definition: **zmap**

An Internet-wide scanner, built to sweep the whole IPv4 address space rather than a handful of hosts. It sends one kind of probe to every address in a range and prints the addresses that reply, so one run asks exactly one question. `-M <module>` chooses the probe: the default module is `tcp_synscan`, which sends a TCP SYN to a port, and `icmp_echoscan` sends ICMP echo requests instead. The argument after the module is the range to sweep, in CIDR form.

Send rate is the parameter that matters most in a scanner, and all three in this lab expose one. `-r <pps>` sets **zmap**'s rate in packets per second and `-B <bps>` sets it in bits per second, accepting the suffixes K, M and G. Given neither flag, **zmap** does not send as fast as the interface allows: it substitutes a built-in default of 10,000 packets per second, so an unflagged run is bounded by that default rather than by the link. The rate chosen decides how long a scan of a given size takes, how much of the link it consumes, and how conspicuous it is to anything watching the traffic.

1. **Sweep the range.** **Host discovery** comes first: it produces the address list every later step works from. Run **zmap** against the prefix with its ICMP echo module, overriding the default TCP SYN probe, and let its progress output print rather than hiding it, because you need the rate it reports. `cd` first into `/root/scan`, an empty directory the lab leaves for whatever you produce:

```
# 104_L4_LAB_attacker: your machine, and the only one you type on
cd /root/scan
zmap -M icmp_echoscan 104.1.0.0/24
```

Addresses that replied are printed one per line. The lines beginning with a date are **zmap** logging what it is doing, and none of them carries a rate. The rate is on the progress lines, which begin with an elapsed time such as `0:03`: a running figure first, then the run's average marked **avg**. The last progress line replaces the running figure with **done** and leaves the average.

Question 1. Record the addresses **zmap** reported. State how many addresses it probed, how many replied, and the percentage of the prefix that replied. Your probe count and the number of addresses in the prefix that could be assigned to an interface are not the same number; state both and account for the difference.

Question 2. Read the average send rate **zmap** reported for this run, in packets per second. The IPv4 unicast space is roughly 3.7 billion addresses once the reserved ranges are removed. At the rate you measured, calculate how long one sweep of the whole space would take, and give the answer in days. Then give one reason the real figure would be worse than your calculation, and name the two **zmap** flags that would let you set the rate deliberately rather than leave the default in place.

Definition: Redirection, pipes, and the file commands around them

A command has two output streams. *Standard output*, numbered 1, carries the data the command was run to produce. *Standard error*, numbered 2, carries its diagnostics. Which stream a tool uses for progress reporting is that tool's own choice and not a rule: `zmap` writes its progress lines to standard error, while other tools print progress on standard output and keep standard error for failures only. `cmd > file` sends standard output to a file, replacing whatever was there; `>>` appends to it instead. `2>` redirects standard error, and `/dev/null` is the destination that discards anything written to it, so `2>/dev/null` suppresses a command's diagnostics without touching its results. `cmd1 | cmd2` feeds the standard output of one command into the input of the next, and `cmd | tee file` passes it on while writing a copy to a file. Three commands read a file back: `cat` prints the whole of it, `head -5` prints its first five lines, and `wc -l` counts its lines.

2. **Keep the result.** **Redirect** the addresses to a file so later steps can read them. `zmap` writes its progress to standard error and its results to standard output, so discarding the first leaves the file holding nothing but addresses:

```
# 104_L4_LAB_attacker: in the shell you opened
zmap -M icmp_echoescan 104.1.0.0/24 2>/dev/null > ips.txt
cat ips.txt
```

Definition: nmap

A scanner built to ask many questions of a few hosts, where `zmap` and `masscan` are built to ask one question of many. It sets its own send rate from how the target responds; `--min-rate` and `--max-rate` override that in packets per second. The flags this lab uses: `-sn` does host discovery only and scans no ports; `-n` suppresses reverse DNS lookups, the PTR-record queries `nmap` would otherwise send the resolver to find a name attached to each address, which are the reverse of the A-record lookup that turns a name into an address; `-PE` restricts discovery to ICMP echo requests; `-Pn` skips discovery altogether and scans every address it is given; `-iL <file>` reads targets from a file; `-p <ports>` sets which ports to scan; `-sV` interrogates each open port to identify the software; `--script <names>` runs scripts against a service. `-oG <file>` writes grepable output, and a bare `-` in place of a filename means standard output, so it can be piped onwards.

Definition: awk

A small language for pulling fields out of lines. A program is a pattern followed by an action in braces, and the action runs on each line the pattern matches. In `awk '/Up$/{print $2}'`, the pattern `/Up$` is a regular expression matching lines that end in `Up` (`$` anchors a match to the end of a line), and `{print $2}` prints the second whitespace-separated field of those lines. The single quotes stop the shell reading the program as its own.

3. **Sweep the same prefix with a second tool.** `nmap` performs host discovery as well, and its default sends several probe types per address rather than one. Run it twice. The first run lets it choose its own probes; the second restricts it to ICMP echo requests with `-PE`, which is the same question `zmap` was asked:

```
# 104_L4_LAB_attacker
nmap -sn -n 104.1.0.0/24 -oG -
nmap -sn -n -PE 104.1.0.0/24 -oG -
```

Grepable output is designed to be fed to another command rather than read. A discovery run writes one line per host; a port scan writes two for each, a **Status:** line and a **Ports:** line, which is what the port-scan step in Part 2 produces. Chain this run into **awk** to pull out just the addresses, in the one-per-line form **zmap** printed them:

```
# 104_L4_LAB_attacker
nmap -sn -n 104.1.0.0/24 -oG - | awk '/Up$/{print $2}'
```

Question 3. The two **nmap** runs do not report the same number of hosts. Give both counts, name the address that appears in one and not the other, and say which of the two runs found it.

4. **Account for the difference between the two counts.** An ICMP echo sweep tests one property: whether the address returns an ICMP echo reply. RFC 1122 states that property as a requirement rather than a courtesy: "Every host **MUST** implement an ICMP Echo server function that receives Echo Requests and sends corresponding Echo Replies." A requirement in a specification is not a prediction of behaviour, though. A host departs from this one whenever its own stack is configured to ignore echo requests, or a firewall in front of it drops them, and it stays reachable on every other protocol and port while doing so.

nmap's default probe set does not depend on that single property. Run by a privileged user, **-sn** sends four probes to each address: an ICMP echo request, a TCP SYN to port 443, a TCP ACK to port 80, and an ICMP timestamp request. A reply to any one of them is enough for **nmap** to report a host. RFC 9293 requires a TCP endpoint with no listener on the target port to reply with a reset rather than stay silent, so a port with nothing behind it can still disclose the host, subject to the same qualification: the segment has to reach a stack that implements the requirement, past anything configured to drop it.

The rule underneath both cases is that a host may reply on one protocol, or on one port, and stay silent on another, in whatever combination its configuration produces. Liveness is not a property a scan reads off a host; it is a conclusion about the specific probe that was sent.

Question 4. Using the rule above, explain why the output of an ICMP echo sweep is a lower bound on the number of hosts present in a prefix rather than a count of them. Then explain how a TCP probe may establish that a host is present even when it replies to no ICMP at all, and name one circumstance in which the TCP probe would fail to establish it either.

Definition: masscan

An Internet-scale port scanner carrying its own packet engine and its own TCP/IP stack rather than using the kernel's. Like **zmap** it sweeps an address range with a single probe type, and the probe is a TCP SYN, the same one **zmap**'s default module sends; it prints one line per open port. **-p21,23,80** is the port list, comma-separated and attached to the flag. **--rate** sets the send rate in packets per second, and the default is 100, so any figure above that raises the rate rather than limiting it. **--wait** is how many seconds it keeps listening after the last probe goes out, because replies are still arriving once sending has finished; the default is 10, and the countdown appears in the progress line as **waiting N-secs**. Set it too low and slow hosts are missed, which is the trade a higher rate always carries.

5. **Sweep with the third tool.** **masscan** sweeps an address range as **zmap** does, and this run asks a different question from the ICMP sweep: not whether an address replies at all, but whether it has a listener on one of three named ports. Give it those three ports across all 256 addresses, raise its rate from the default 100 to 1000 packets per second, and let it listen for 3 seconds after the last probe:

```
# 104_L4_LAB_attacker
masscan -p21,23,80 104.1.0.0/24 --rate 1000 --wait 3
```

Question 5. You have now swept the same prefix with three tools sending different probes. Using your own results, state which addresses the **masscan** run identified that the ICMP echo sweep did not, and explain from the probe types why the two disagree. Then name the one property of each open port that none of the three runs has established, and which of the three tools Part 2 uses to establish it.

Part 2: what each host is running

An address that replies is not yet a target. What can be attacked is a service, so the second pass moves from addresses to the processes listening on them. It has two stages. The first sends a TCP SYN to each port in a list and classifies the port from the reply, which establishes where a listener exists. The second sends a series of application-layer probes to each port that has one and matches the replies against a database of known service signatures, which establishes what that listener is and, often, which version of it.

Definition: Port states: open, closed, filtered

What a scanner concludes from how a port replied. *Open* means something accepted the connection. *Closed* means the host replied but no process is listening on that port; a TCP endpoint with no listener typically replies with a reset, which shows the host is present and the port is empty. *Filtered* is the scanner's verdict when nothing came back at all, which is what a firewall dropping the probe looks like, and also what an address with no host on it looks like. Open and closed are both replies from the port itself; filtered is the absence of one, and it is the state that carries the least information.

6. **Scan the ports of every host you found.** **-iL** makes the list you built in Part 1 the input here, and **-Pn** scans every address on it whether or not it replied to a ping, which matters given what you found in Part 1. **tee** shows the output and saves it at the same time:

```
# 104_L4_LAB_attacker
nmap -Pn -n -iL ips.txt -oG - | tee report.txt
```

That scans `nmap`'s default set of 1000 ports per host. Note that `ips.txt` came from the ping sweep, so it is missing a host; add the one you identified in Question 3 by hand before you go on:

```
# 104_L4_LAB_attacker
echo <the address the ping sweep missed> >> ips.txt
nmap -Pn -n -iL ips.txt -oG - | tee report.txt
```

Definition: `grep`

Prints only the lines of its input that match a pattern, and drops the rest. The pattern comes first, then the file to read, or nothing if the input arrives through a pipe. `'^Host:'` matches lines beginning with `Host:`, because `^` anchors a match to the start of a line; a plain word like `Ports` matches anywhere in the line. The single quotes stop the shell from touching the pattern before `grep` sees it.

Two `grep` passes reduce that to the lines carrying port data: the first keeps the per-host lines, and the second keeps those of them that list ports rather than status:

```
# 104_L4_LAB_attacker
grep '^Host:' report.txt | grep Ports
```

Question 6. For every address in your list, record which ports are open. Some of them will have none; for each of those, quote what `nmap` reports about its 1000 scanned ports, and say what that tells you about whether the host is running at all.

Question 7. Apply the **port state** definitions to your own results. Take the addresses that had nothing open and the state you quoted their 1000 ports coming back in for Question 4. Explain how that output distinguishes those addresses from an address in the prefix with no host on it at all, and describe what the same scan would have printed instead if the router in front of them had silently dropped your probes.

Definition: Service banner

The text a service sends when a client first connects, before any request is made. An FTP server greets a new connection with a status line beginning `220`, the FTP code for a service ready for a new user. What follows the code is free text the administrator sets, and the default in most distributions names the software and its version. A banner is the cheapest way to identify what is running, because reading it takes no more than opening a connection, and it is also the cheapest thing to change.

7. **Identify the software, not just the port.** A port number is a registry entry, not a guarantee, and it fails in both directions. Port 80 is registered to HTTP, but nothing stops a program that is not an HTTP server from listening there, and nothing stops an HTTP server from listening on 54321 instead; the second case is why a scan of the default 1000 ports is a sample of a host rather than a survey of it. `-sV` makes `nmap` probe each open port and report what replied. Run it against each of the three ports you found open, replacing the address and port:

```
# 104_L4_LAB_attacker: run this once per open port you found
nmap -Pn -n -sV -p <port> <address>
```

Question 8. Record what `-sV` reported for all three services. Two of them come back with a software name and an exact version number; the third comes back with a service name and no version. Name which is which, then say what the two identified services put on the wire that the third does not, and name one change an administrator could make to those two that would leave `-sV` with as little to go on as the third.

Two of those runs return in about a second. The third takes tens of seconds and ends by printing a block of escaped bytes that fills most of a screen, starting `SF-Port23-TCP:.` Both follow from the same cause: no signature matched an early reply, so `nmap` worked through the rest of its probe list and then printed the entire exchange.

Definition: Service fingerprint (the SF: block)

What `-sV` prints when a service replied to its probes but matched nothing in `nmap`'s signature database. `nmap` sends a fixed series of probes to an open port and compares each reply against thousands of stored signatures. When none matches, it prints the exchange itself, wrapped across continuation lines that each begin `SF:`, and gives a URL for submitting it so that a later release can name the service. Each `%r(Probe,Length,"...")` entry inside the block is one probe by name, the length of the reply to it in hexadecimal, and the reply itself with every unprintable byte escaped, most of them as `\xNN`. Carriage return, line feed, tab and NUL have their own shorter escapes, `\r`, `\n`, `\t` and `\0`.

That block is the **service fingerprint**. It is diagnostic output rather than an error, and nothing in it is a command to run. The probe names record what was sent: `NULL` sends nothing at all and only listens, `GetRequest` sends an HTTP request, `SIPOptions` sends an OPTIONS request in the Session Initiation Protocol, the signalling protocol used to set up voice calls. The quoted string beside each name is what this service sent in reply, so the block is the evidence behind the one-line verdict above it.

Definition: Telnet and its option negotiation

Telnet is a remote terminal protocol specified in its current form by RFC 854 in 1983, on TCP port 23, carrying everything in the clear. RFC 854 says option requests are "likely to flurry back and forth when a TELNET connection is first established", and each side may also renegotiate mid-session, so the opening exchange is the usual case rather than a rule. The options settled are things such as who echoes typed characters. Each step is three bytes: `\xFF`, the IAC byte, meaning "interpret as command"; then a command, `\xFB WILL` or `\xFD DO` to offer or request an option, `\xFC WON'T` or `\xFE DON'T` to refuse; then the option number, for example `\x01` for echo and `\x03` for suppress-go-ahead. The telnet server in this lab sends its opening negotiation as soon as the connection is up, before any prompt; RFC 854 does not require that, and another server may wait.

Every reply in the fingerprint opens with the same run of `\xFF` triples, because that **option negotiation** is the first thing this service says on any connection, whatever the probe that opened it.

Definition: Nmap Scripting Engine (NSE)

A library of scripts **nmap** can run against a service once it knows the service is there. Each script asks one specific question: **http-title** fetches a page and reports its title, **ftp-anon** tries to log in without an account, **banner** simply prints whatever the service says first. They are named after **--script**: one name runs one script, a comma-separated list runs several against the same port in one pass, and a wildcard runs a whole family at once.

8. **Ask service-specific questions.** **NSE** scripts turn "port 80 is open" into a description of the site. Run three of them against the HTTP service at once:

```
# 104_L4_LAB_attacker
nmap -Pn -n -p 80 --script http-title,http-server-header,http-enum <the HTTP host>
```

Question 9. Record what each of the three scripts returned. For each one, state the single fact it added that you did not already have from the port number and the **-sV** output.

9. **Grab the banners.** The generic **banner** script prints whatever a service sends on connect. Run it against the FTP port and then the telnet port:

```
# 104_L4_LAB_attacker
nmap -Pn -n -p 21 --script banner <the FTP host>
nmap -Pn -n -p 23 --script banner <the telnet host>
```

One of the two returns readable text. The other returns a short run of escaped bytes beginning `\xFF`: the option negotiation and nothing after it, the same triples that open every reply in the fingerprint block. The **banner** script reports what arrives first and stops there, so on this service that is all it has to report.

Question 10. You have two services and only one returned a useful banner. Explain why the negotiation bytes tell an attacker much less about the telnet service than the FTP greeting told them about the FTP service, and name one thing you could do by hand to learn more about the telnet service than **--script banner** managed.

Definition: Anonymous FTP

A deliberate FTP configuration in which the server keeps an account literally named **anonymous** that anybody may log into, traditionally accepting any string as the password, though many sites ask for an e-mail address instead. RFC 1635 is an informational memo rather than a standard, and it describes the conventional account as able only to log in, list a restricted set of directories, and retrieve files. Those limits come from how the site configures its server, not from the RFC: vsftpd's **anon_upload_enable** lifts them. The account exists so a site can publish files to the public without issuing accounts.

10. **Try the FTP service without an account.** **ftp-anon** attempts an **anonymous FTP** login and reports what it finds:

```
# 104_L4_LAB_attacker
nmap -Pn -n -p 21 --script ftp-anon <the FTP host>
```

Definition: curl

A command-line client that makes one request and prints the response. The scheme at the front of the URL picks the protocol, so `ftp://` makes it an FTP client rather than an HTTP one. `--user <name>:<password>` supplies credentials, split on the first colon, and leaving the half after the colon empty sends an empty password rather than prompting for one. An empty password is not what anonymous FTP asks for; the account conventionally takes any string, often an e-mail address, and an empty one works here because this lab's server accepts it. A URL ending in `/` names a directory and `curl` lists it; a URL ending in a filename retrieves that file.

If the login is allowed the script says so, quotes the FTP status code the server returned, and lists what it can see. Then go and read it yourself with `curl`, which `ftp-anon` does not do for you:

```
# 104_L4_LAB_attacker
curl --user "anonymous:" ftp://<the FTP host>/pub/      # list the
    directory
curl --user "anonymous:" ftp://<the FTP host>/pub/<file>  # read what is in
    it
```

Question 11. Anonymous FTP is a supported feature, not a bug: RFC 1635 describes it as "a means by which archive sites allow general access to their archives", while noting that not every site offering FTP permits anonymous transfers. State what you found in this server's share, and use it to argue whether "anonymous FTP is enabled" is by itself a finding worth reporting. Be specific about what would have to be true for the answer to go the other way.

11. **Check for known vulnerabilities.** `nmap` carries a family of scripts that test for published vulnerabilities in a service. Run the HTTP family, quoting the pattern so the shell hands the `*` to `nmap` instead of expanding it against the files in the current directory:

```
# 104_L4_LAB_attacker
nmap -Pn -n -p 80 --script "http-vuln-*" <the HTTP host>
```

Question 12. Record what this returned. Explain why that result is not evidence that the HTTP service has no vulnerabilities, and state what you would do next with the exact version string `-sV` gave you in Question 8.

Part 3: from enumeration to access

Nothing so far has touched a login. The scan produced an account name, a host, and a service that takes a password, which between them are enough to attack a login without any exploit at all.

Definition: Dictionary attack

Guessing a password by working through a prepared list of likely ones rather than every possible combination. The list is what makes it feasible: drawn from the 95 printable ASCII characters, an eight-character password has 95^8 , or about 6.6×10^{15} , combinations, which no attacker enumerates over a network. A few thousand passwords account for a large share of real accounts, because people reuse the same ones, so a list of a few thousand is the shortcut. Your machine carries a fifty-entry list at `/usr/share/minilabs/passwords.txt`.

12. **Use what the enumeration gave you.** The file you read out of the anonymous FTP share names an account and the host it is used on. Confirm the service on that host is the telnet one you already found open, and look at the password list you are about to use:

```
# 104_L4_LAB_attacker
wc -l /usr/share/minilabs/passwords.txt
head -5 /usr/share/minilabs/passwords.txt
```

Question 13. State the account name and the host address you took from the FTP share. Then explain what this step gave you that no amount of port scanning would have: be specific about which of the two values you now have could not have come from a scan of the telnet host itself.

Definition: hydra

A login guesser. It opens a connection to a service, tries one username and password pair, and moves on to the next. `-l` gives it a single username, where `-L` would take a file of them, and `-P` points at a file of passwords, where `-p` would take a single one. `-t 4` runs four attempts in parallel, and `-f` stops the moment a pair works instead of finishing the list. The target is written `service://host`, and the service half chooses which module `hydra` loads to speak the protocol, here `telnet`.

13. **Guess the password.** Point `hydra` at the account you recovered and the fifty-entry list:

```
# 104_L4_LAB_attacker
hydra -l <the account> -P /usr/share/minilabs/passwords.txt -t 4 -f
telnet://<the telnet host>
```

That is a **dictionary attack**. Run it once more behind `time`, which reports how long the command it is given took, so the next question has a figure to work from:

```
# 104_L4_LAB_attacker
time hydra -l <the account> -P /usr/share/minilabs/passwords.txt -t 4 -f
telnet://<the telnet host>
```

Question 14. Record the password `hydra` recovered and how long the run took. Then calculate: if the service had instead delayed five seconds after every failed attempt, and `hydra` still ran four attempts in parallel, roughly how long would the same fifty-entry list have taken? Show the arithmetic.

14. **Log in.** Use the credentials by hand and confirm they work. The `telnet` client takes the host to connect to and nothing else, because port 23 is its default:

```
# 104_L4_LAB_attacker
telnet <the telnet host>
```

There is a file in the account's home directory. Read it, then leave the session open for the next step or log back in when you need it.

Question 15. Try one login with the correct account name and a wrong password, and watch what the server does after it refuses you. Explain how that behaviour affects an attacker guessing passwords, and contrast it with what a lockout after three failures would have cost them.

Definition: tcpdump

Prints the packets crossing an interface as they arrive. `-i <interface>` picks the interface to listen on, `-n` leaves addresses and ports as numbers instead of looking up names, and `-A` prints each packet, minus its link-layer header, as ASCII underneath the header line, so the IP and TCP header bytes appear as unreadable characters before the readable payload. The quoted expression at the end is a filter, and only packets matching it are shown: `tcp port 23` matches TCP segments with 23 as either the source or the destination port, so both directions of the session appear. The quotes keep the shell from splitting the expression into separate arguments.

15. **Watch the credentials cross the wire.** Telnet applies no encryption to anything it carries, so every byte of that session crossed the link exactly as it was typed. Open a second terminal on your own machine and start a **tcpdump** capture, then log in again in the first:

```
# 104_L4_LAB_attacker: a SECOND terminal on your machine
docker exec -it 104_L4_LAB_attacker bash
tcpdump -n -i 104-ext -A 'tcp port 23'
```

```
# 104_L4_LAB_attacker: back in your first terminal
telnet <the telnet host>
```

Question 16. Find the account name and the password in the capture and quote the lines they appear on. Explain what an attacker sitting on the router between the two hosts, who had captured this session without doing any scanning at all, would have obtained.

Question 17. Three separate weaknesses had to line up for this to work: the protocol carrying the login, the password itself, and how the service treats failed attempts. Name the specific weakness in each case, then say which single one you would fix first and justify the choice in terms of what an attacker still could and could not do afterwards.

Question 18. Replacing telnet with SSH is the change usually recommended first. Of the three weaknesses you named in Question 17, state which ones switching to SSH would fix and which would survive unchanged, and explain why.

Checks: what “done” means

- **Part 1:** you have a file of addresses that includes the host the ping sweep missed, and you can say which probe found it.

- **Part 2:** you have, for every address, either a list of open ports or a statement that it has none; and for each open port, the software and version behind it.
- **Part 3:** you can log in over telnet with credentials you recovered rather than were given, and you have seen both of them in a packet capture.

`scripts/status.sh` confirms every service in the lab is running and reachable from your machine, so you can tell a broken lab from an empty scan result. It does not list the addresses or ports, because those are what you are being asked to find. `scripts/reset.sh` deletes your scan output and restarts the services, so a part can be run again from nothing.

Safety. Everything here lives on one isolated router and switch with no route off the host. Every probe, guess and capture only ever reaches the lab's own containers. Running these same tools against a network you do not own or have written permission to test is an offence in most jurisdictions, including under the Crimes Act 1961 in New Zealand, and it is a breach of the acceptable-use policy of every university and consumer internet provider.

When you are done, tear the lab down. Choose **Teardown** in the TUI, or run `scripts/teardown.sh` directly, to remove the containers and their wiring so nothing is left running on your host. **Reset to baseline** only clears your scan output; only **Teardown** deletes the lab.